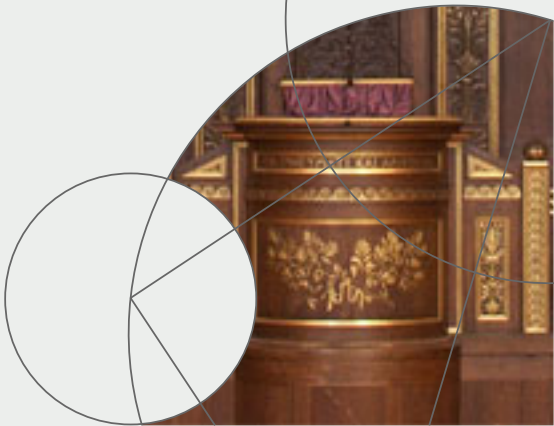
STEVENS INSTITUTE OF TECHNOLOGY

Data Acquisition and Processing I: Big Data

Rensheng Wang,
rwang1@stevens.edu
Dept. of Electrical and Computer Engineering
Stevens Institute of Technology



Recommendation Systems

Broadly speaking, recommender systems are based on one of two strategies.

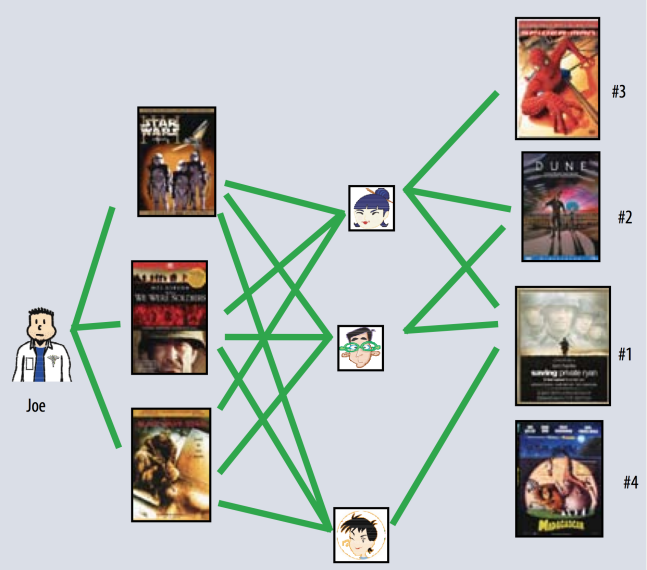
- The content filtering approach creates a profile for each user or product to characterize its nature.
  - For example, a movie profile could include attributes regarding its genre, the participating actors, its box office popularity, and so forth. User profiles might include demographic information or answers provided on a suitable questionnaire.
  - The profiles allow programs to associate users with matching products. Of course, content-based strategies require gathering external information that might not be available or easy to collect.
- The collaborative filtering analyzes relationships between users and interdependencies among products to identify new user-item associations.
- The two primary areas of collaborative filtering are
  - the neighborhood methods
  - the latent factor models

Neighborhood methods

- Neighborhood methods are centered on computing the relationships between items or, alternatively, between users.
- The item-oriented approach evaluates a user's preference for an item based on ratings of "neighboring" items by the same user.
- A product's neighbors are other products that tend to get similar ratings when rated by the same user.
- For example, consider the movie Saving Private Ryan. Its neighbors might include war movies, Spielberg movies, and Tom Hanks movies, among others. To predict a particular user's rating for Saving Private Ryan, we would look for the movie's nearest neighbors that this user actually rated.

Neighborhood methods

- As Figure 1 illustrates, the user-oriented approach identifies like-minded users who can complement each other's ratings.

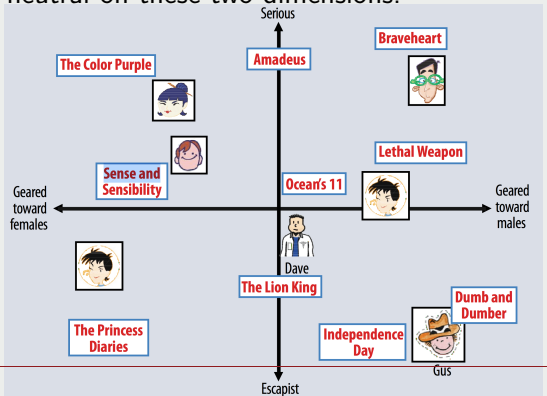


Latent factor methods

- Latent factor models are an alternative approach that tries to explain the ratings by characterizing both items and users on, say, 20 to 100 factors inferred from the ratings patterns.
- In a sense, such factors comprise a computerized alternative to the aforementioned human-created song genes.
- For movies, the discovered factors might measure obvious dimensions such as comedy versus drama, amount of action, or orientation to children; Less well-defined dimensions such as depth of character development or quirkiness; or completely uninterpretable dimensions.
- For users, each factor measures how much the user likes movies that score high on the corresponding movie factor.

Latent factor methods

- Figure 2 illustrates this idea for a simplified example in two dimensions. Consider two hypothetical dimensions characterized as female- versus male-oriented and serious versus escapist. The figure shows where several well-known movies and a few fictitious users might fall on these two dimensions. For this model, a user's predicted rating for a movie, relative to the movie's average rating, would equal the dot product of the movie's and user's lo- cations on the graph. For example, we would expect Gus to love Dumb and Dumber, to hate The Color Purple, and to rate Braveheart about average. Note that some movies-for example, Ocean's 11-and users-for example, Dave would be characterized as fairly neutral on these two dimensions.



Matrix Factorization Methods

- Some of the most successful realizations of latent factor models are based on matrix factorization. In its basic form, matrix factorization characterizes both items and users by vectors of factors inferred from item rating patterns. High correspondence between item and user factors leads to a recommendation. These methods have become popular in recent years by combining good scalability with predictive accuracy. In addition, they offer much flexibility for model- ing various real-life situations.
- Recommender systems rely on different types of input data, which are often placed in a matrix with one dimension representing users and the other dimension representing items of interest. The most convenient data is high-quality explicit feedback, which includes explicit input by users regarding their interest in products. For example, Netflix collects star ratings for movies, and TiVo users indicate their preferences for TV shows by pressing thumbs-up and thumbs-down buttons. We refer to explicit user feedback as ratings. Usually, explicit feedback com- prises a sparse matrix, since any single user is likely to have rated only a small percentage of possible items.

Matrix Factorization Methods

- One strength of matrix factorization is that it allows incorporation of additional information. When explicit feedback is not available, recommender systems can infer user preferences using implicit feedback, which indirectly reflects opinion by observing user behavior including purchase history, browsing history, search patterns, or even mouse movements. Implicit feedback usually denotes the presence or absence of an event, so it is typically represented by a densely filled matrix.

A Basic Matrix Factorization Method

- Matrix factorization models map both users and items to a joint latent factor space of dimensionality f, such that user-item interactions are modeled as inner products in that space.
- Accordingly, each item i is associated with a vector $\mathbf{q}_i \in \mathbb{R}^f$, and each user u is associated with a vector $\mathbf{p}_u \in \mathbb{R}^f$.
- For a given item i, the elements of $\mathbf{q}_i$ measure the extent to which the item possesses those factors, positive or negative. For a given user u, the elements of $\mathbf{p}_u$ measure the extent of interest the user has in items that are high on the corresponding factors, again, positive or negative.
- The resulting dot product, $\mathbf{q}_i^T \mathbf{p}_u$, captures the interaction between user u and item iâĂŤthe user's overall interest in the item's characteristics. This approximates user u's rating of item i, which is denoted by r_{ui}, leading to the estimate

$$\hat{r}_{ui} = \mathbf{q}_i^T \mathbf{p}_u \tag{1}$$

A Basic Matrix Factorization Method

- The major challenge is computing the mapping of each item and user to factor vectors $\mathbf{q}_i,\ \mathbf{p}_u \in \mathbb{R}^f$.
- After the recommender system completes this mapping, it can easily estimate the rating a user will give to any item by using aforementioned Equation (1).
- Such a model is closely related to singular value decomposition (SVD), a well-established technique for identifying latent semantic factors in information retrieval.
- Applying SVD in the collaborative filtering domain requires factoring the user-item rating matrix.

Matrix Factorization Method vs. SVD

- SVD for recommendation

A Basic Matrix Factorization Method

- This often raises difficulties due to the high portion of missing values caused by sparseness in the user-item ratings matrix. Conventional SVD is undefined when knowledge about the matrix is incomplete.
- Moreover, carelessly addressing only the relatively few known entries is highly prone to overfitting.
- Earlier systems relied on imputation to fill in missing ratings and make the rating matrix dense. However, imputation can be very expensive as it significantly increases the amount of data.
- In addition, inaccurate imputation might distort the data considerably. Hence, more recent works suggested modeling directly the observed ratings only, while avoiding overfitting through a regularized model.

A Basic Matrix Factorization Method

- To learn the factor vectors ($\mathbf{q}_i^T \mathbf{p}_u$), the system minimizes the regularized squared error on the set of known ratings:

$$\arg\min_{\mathbf{q}^*, \mathbf{p}^*} \sum_{u,i,\in\kappa} \left(\hat{r}_{ui} - \mathbf{q}_i^T \mathbf{p}_u\right)^2 + \lambda \left(\|\mathbf{q}_i\|^2 + \|\mathbf{p}_u\|^2\right) \tag{2}$$

  where κ is the set of (u, i) pairs for which r_{ui} is known (the training set).

- The system learns the model by fitting the previously observed ratings.
- However, the goal is to generalize those previous ratings in a way that predicts future, unknown ratings.
- Thus, the system should avoid overfitting the observed data by regularizing the learned parameters, whose magnitudes are penalized.
- The constant λ controls the extent of regularization and is usually determined by cross-validation.

Learning Algorithm - stochastic gradient descent

Two approaches to minimizing Equation (2) are stochastic gradient descent and alternating least squares (ALS).

- Stochastic gradient descent

  The algorithm loops through all ratings in the training set. For each given training case, the system predicts r_{ui} and computes the associated prediction error.

$$e_{ui} \triangleq r_{ui} - \mathbf{q}_i^T \mathbf{p}_u$$

  Then it modifies the parameters by a magnitude proportional to γ in the opposite direction of the gradient, yielding:

$$\begin{aligned} \mathbf{q}_i &\leftarrow \mathbf{q}_i + \gamma(e_{ui}\mathbf{p}_u - \lambda \mathbf{q}_i) \\ \mathbf{p}_u &\leftarrow \mathbf{p}_u + \gamma(e_{ui}\mathbf{q}_i - \lambda \mathbf{p}_u) \end{aligned}$$

- This popular approach combines implementation ease with a relatively fast running time. Yet, in some cases, it is beneficial to use ALS optimization.

Learning Algorithm - alternating least squares (ALS)

- Because both $\mathbf{q}_i$ and $\mathbf{p}_u$ are unknowns, Equation (2) is not convex. However, if we fix one of the unknowns, the optimization problem becomes quadratic and can be solved optimally.
- Thus, ALS techniques rotate between fixing the $\mathbf{q}_i$'s and fixing the $\mathbf{p}_u$'s. When all $\mathbf{p}_u$'s are fixed, the system recomputes the $\mathbf{q}_i$'s by solving a least-squares problem, and vice versa. This ensures that each step decreases Equation (2) until convergence.
- While in general stochastic gradient descent is easier and faster than ALS, ALS is favorable in at least two cases. The first is when the system can use parallelization. In ALS, the system computes each $\mathbf{q}_i$ independently of the other item factors and computes each $\mathbf{p}_u$ independently of the other user factors. This gives rise to potentially massive parallelization of the algorithm.

  The second case is for systems centered on implicit data. Because the training set cannot be considered sparse, looping over each single training caseâĂŤas gradient descent doesâĂŤwould not be practical. ALS can efficiently handle such cases.

Adding Biases

- One benefit of the matrix factorization approach to collaborative filtering is its flexibility in dealing with various data aspects and other application-specific requirements. This requires accommodations to Equation (1) while staying within the same learning framework.
- Equation (1) tries to capture the interactions between users and items that produce the different rating values. However, much of the observed variation in rating values is due to effects associated with either users or items, known as biases or intercepts, independent of any interactions.
- For example, typical collaborative filtering data exhibits large systematic tendencies for some users to give higher ratings than others, and for some items to receive higher ratings than others. After all, some products are widely perceived as better (or worse) than others.
- Thus, it would be unwise to explain the full rating value by an interaction of the form $\mathbf{q}_i^T \mathbf{p}_u$. Instead, the system tries to identify the portion of these values that individual user or item biases can explain, subjecting only the true interaction portion of the data to factor modeling.

Adding Biases

- A first-order approximation of the bias involved in rating r_{ui} is as follows:

$$b_{ui} = \mu + b_i + b_u \tag{3}$$

- The bias involved in rating r_{ui} is denoted by b_{ui} and accounts for the user and item effects. The overall average rating is denoted by μ; the parameters b_u and b_u indicate the observed deviations of user u and item i, respectively, from the average.
- For example, suppose that you want a first-order estimate for user Joe's rating of the movie Titanic. Now, say that the average rating over all movies, μ, is 3.7 stars. Furthermore, Titanic is better than an average movie, so it tends to be rated 0.5 stars above the average. On the other hand, Joe is a critical user, who tends to rate 0.3 stars lower than the average. Thus, the estimate for Titanic's rating by Joe would be 3.9 stars (3.7 + 0.5 - 0.3).
- Biases extend Equation (1) as follows:

$$\hat{r}_{ui} = \mu + b_i + b_u + \mathbf{q}_i^T \mathbf{p}_u \tag{4}$$

Adding Biases

- The observed rating is broken down into its four components: global average, item bias, user bias, and user- item interaction. This allows each component to explain only the part of a signal relevant to it.
- The system learns by minimizing the squared error function

$$\arg\min_{\mathbf{q}^*,\mathbf{p}^*,b^*} \sum_{u,i,\in\kappa} \left(\hat{r}_{ui} - \mu - b_u - b_i - \mathbf{q}_i^T \mathbf{p}_u\right)^2 + \lambda\left(\|\mathbf{q}_i\|^2 + \|\mathbf{p}_u\|^2 + b_u^2 + b_i^2\right) \tag{5}$$

Since biases tend to capture much of the observed signal, their accurate modeling is vital. Hence, other works offer more elaborate bias models

Implementation of Matrix Factorization Method

- In a recommendation system such as Netflix or MovieLens, there is a group of users and a set of items (movies for the above two systems). Given that each users have rated some items in the system, we would like to predict how the users would rate the items that they have not yet rated, such that we can make recommendations to the users. In this case, all the information we have about the existing ratings can be represented in a matrix. Assume now we have 5 users and 4 items, and ratings are integers ranging from 1 to 5, the matrix may look something like this (a hyphen means that the user has not yet rated the movie):

Users	D 1	D 2	D 3	D 4
User 1	5	3	-	1
User 2	4	-	-	1
User 3	1	1	-	5
User 4	1	-	-	4
User 5	-	1	5	4

Table: User ratings on different items

Implementation of Matrix Factorization Method

- Let us assume we have U users and D items. Denote the matrix $\mathbf{R}_{U\times D}$ be the matrix containing all the users and all the related item ratings.
- We like to find the K latent features or factors. In other words, we would like to find 2 matrices $\mathbf{P}_{U\times K}$ and $\mathbf{Q}_{D\times K}$ such that their product approximates the matrix $\mathbf{R}_{U\times D}$

$$\mathbf{R} \approx \mathbf{P}\mathbf{Q}^T = \hat{\mathbf{R}}$$

- This can be obtained by SVD decomposition with K eigenvectors associated with the largest K eigenvalues.
- In this way, each row of $\mathbf{P}$ would represent the strength of the associations between a user and the features. Similarly, each row of $\mathbf{Q}$ would represent the strength of the associations between an item and the features.
- To get the prediction of a rating of an item d_j by u_i, we can calculate the dot product of the two vectors corresponding to $u_i$and d_j:

$$\hat{r}_{ij} = \mathbf{p}_i^T \mathbf{q}_j = \sum_{k=1}^{k} p_{ik} \cdot q_{kj}$$

Implementation of Matrix Factorization Method

- The difference here, usually called the error between the estimated rating and the real rating, can be calculated by the following equation for each user-item pair:

$$\hat{e}_{ij}^2 = (r_{ij} - \hat{r}_{ij})^2 = \left(r_{ij} - \mathbf{p}_i^T \mathbf{q}_j\right)^2 = \left(r_{ij} - \sum_{k=1}^{k} p_{ik} \cdot q_{kj}\right)^2$$

- Here we consider the squared error because the estimated rating can be either higher or lower than the real rating.
- To minimize the error, we have to know in which direction we have to modify the values of p_{ik} and q_{kj}. In other words, we need to know the gradient at the current values, and therefore we differentiate the above equation with respect to these two variables separately:

$$\frac{\partial}{\partial p_{ik}} e_{ij}^2 = -2(r_{ij} - \hat{r}_{ij})(q_{kj}) = -2e_{ij}q_{kj}$$

$$\frac{\partial}{\partial q_{ik}} e_{ij}^2 = -2(r_{ij} - \hat{r}_{ij})(p_{ik}) = -2e_{ij}p_{ik}$$

Implementation of Matrix Factorization Method

- Having obtained the gradient, we can now formulate the update rules for both p_{ik} and q_{kj}:

$$p'_{ik} = p_{ik} + \alpha \frac{\partial}{\partial p_{ik}} e_{ij}^2 = p_{ik} + 2\alpha e_{ij} q_{kj}$$
$$q'_{kj} = q_{kj} + \alpha \frac{\partial}{\partial q_{kj}} e_{ij}^2 = q_{kj} + 2\alpha e_{ij} p_{ik}$$

- Here, $\alpha$is a constant whose value determines the rate of approaching the minimum. Usually we will choose a small value for α, say 0.0002. This is because if we make too large a step towards the minimum we may run into the risk of missing the minimum and end up oscillating around the minimum.
- A question might have come to your mind by now: if we find two matrices $\mathbf{P}$ and $\mathbf{Q}$ such that $\mathbf{P} \times \mathbf{Q} \approx \mathbf{R}$, is not that our predictions of all the unseen ratings will all be zeros?
- In fact, we are not really trying to come up with$\mathbf{P}$ and $\mathbf{Q}$ such that we can reproduce $\mathbf{R}$ exactly. Instead, we will only try to minimise the errors of the observed user-item pairs.

Implementation of Matrix Factorization Method

- The above algorithm is a very basic algorithm for factorizing a matrix. There are a lot of methods to make things look more complicated. A common extension to this basic algorithm is to introduce regularization to avoid overfitting. This is done by adding a parameter β and modify the squared error as follows:

$$e_{ij}^2 = (r_{ij} - \sum_{k=1}^{K} p_{ik} q_{kj})^2 + \frac{\beta}{2} \sum_{k=1}^{K} (||\mathbf{P}||^2 + ||\mathbf{Q}||^2)$$

- In other words, the new parameter β is used to control the magnitudes of the user-feature and item-feature vectors such that $\mathbf{P}$ and $\mathbf{Q}$ would give a good approximation of $\mathbf{R}$ without having to contain large numbers. In practice, β is set to some values in the range of 0.02. The new update rules for this squared error can be obtained by a procedure similar to the one described above.

- The new update rules are as follows.

$$p'_{ik} = p_{ik} + \alpha \frac{\partial}{\partial p_{ik}} e_{ij}^2 = p_{ik} + \alpha(2e_{ij} q_{kj} - \beta p_{ik})$$

$$q'_{kj} = q_{kj} + \alpha \frac{\partial}{\partial q_{kj}} e_{ij}^2 = q_{kj} + \alpha(2e_{ij} p_{ik} - \beta q_{kj})$$

Matlab Implementation of Matrix Factorization

- Matlab code to test the matrix factorization:

```
clear all
close all

% R : Rating matrix
R=[ 5 3 0 1;
    4 0 0 1;
    1 1 0 5;
    1 0 0 4;
    0 1 5 4];
[nRow, nCol]=size(R);
% K is the number of factors
K=2;
P=rand(nRow, K);
Q=rand(K, nCol);

steps=5000;
alpha=0.0002;
beta=0.02;


[nP, nQ]=matrix_factorization(R,P,Q,K, steps, alpha, beta);
[nP*nQ]
[R]
```

Matlab Implementation of Matrix Factorization

- Matlab code to test the matrix factorization:

```
>> [R]
[nP*nQ]

R =

     5     3     0     1
     4     0     0     1
     1     1     0     5
     1     0     0     4
     0     1     5     4


ans =

    5.0272    2.8334    5.7201    0.9964
    3.9356    2.2207    4.6045    0.9969
    1.1161    0.6835    4.0319    4.9612
    0.9371    0.5717    3.2742    3.9749
    2.4594    1.4269    4.8650    4.0341

>>
```